

CS 3308: Information Retrieval

Programming Assignment: Unit 3

Instructor: Rupali Memane

Updated code: Retrieval

```
import sys
import os
import re
import math
import sqlite3
import time
from collections import defaultdict

# ===== NEW: Text Processing Utilities =====
# Stop words list (you might want to expand this)
STOP_WORDS = {
    'a', 'an', 'and', 'are', 'as', 'at', 'be', 'by', 'for', 'from',
    'has', 'he', 'in', 'is', 'it', 'its', 'of', 'on', 'that', 'the',
    'to', 'was', 'were', 'will', 'with', 'i', 'you', 'we', 'they',
    'this', 'that', 'these', 'those', 'have', 'has', 'had', 'do',
    'does', 'did', 'but', 'or', 'if', 'because', 'as', 'until',
    'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against',
    'between', 'into', 'through', 'during', 'before', 'after',
    'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out',
    'on', 'off', 'over', 'under', 'again', 'further', 'then',
    'once', 'here', 'there', 'when', 'where', 'why', 'how',
    'all', 'any', 'both', 'each', 'few', 'more', 'most',
    'other', 'some', 'such', 'no', 'nor', 'not', 'only',
    'own', 'same', 'so', 'than', 'too', 'very', 's', 't',
    'can', 'will', 'just', 'don', 'should', 'now'
}

# Porter Stemmer (you can use a library like nltk instead)
class PorterStemmer:
    """A simplified Porter Stemmer for demonstration"""

    def stem(self, word):
        # Basic stemming rules - in practice, use a proper stemmer
        word = word.lower()
        if word.endswith('sses'):
            word = word[:-2]
        elif word.endswith('ies'):
            word = word[:-2]
        elif word.endswith('ss'):
            pass
        elif word.endswith('s'):
            word = word[:-1]
        return word

# Initialize stemmer
stemmer = PorterStemmer()

# ===== MODIFIED: Global counters =====
documents = 0
```

```

tokens_before_filtering = 0 # Total terms parsed from all documents
tokens_after_filtering = 0 # Tokens after processing
unique_terms = 0 # Total unique terms in index
stop_words_matched = 0 # Total stop words removed

# ===== MODIFIED: Data structures =====
# We'll store term frequencies per document and document frequencies
term_doc_freq = defaultdict(lambda: defaultdict(int)) # term -> doc -> tf
term_doc_set = defaultdict(set) # term -> set of documents containing it
all_docs = set() # All document IDs
term_to_id = {} # term -> term_id
id_to_term = {} # term_id -> term

# ===== MODIFIED: Token processing =====
def process_token(token, doc_id):
    global tokens_before_filtering, tokens_after_filtering,
    stop_words_matched

    tokens_before_filtering += 1

    # 1. Convert to lowercase
    token = token.lower()

    # 2. Skip if starts with punctuation
    if token and token[0] in '!"#$%^&*()_+-=[]{}|;: ",.<>?/~`\'":
        return None

    # 3. Skip if it's a stop word
    if token in STOP_WORDS:
        stop_words_matched += 1
        return None

    # 4. Apply stemming
    token = stemmer.stem(token)

    # Skip empty tokens
    if not token:
        return None

    tokens_after_filtering += 1
    return token

# ===== MODIFIED: Parse tokens =====
def parsetoken(line, doc_id):
    global unique_terms

    line = line.replace('\t', ' ')
    line = line.strip()

    # Split on non-word characters
    tokens = re.split(r'\W+', line)

    for token in tokens:
        if not token: # Skip empty tokens
            continue

        # Process the token (filtering, stemming, etc.)
        processed_token = process_token(token, doc_id)

```

```

        if processed_token:
            # Update term frequencies
            term_doc_freq[processed_token][doc_id] += 1
            term_doc_set[processed_token].add(doc_id)

            # Track unique terms
            if processed_token not in term_to_id:
                unique_terms += 1
                term_to_id[processed_token] = unique_terms
                id_to_term[unique_terms] = processed_token

# ===== MODIFIED: Process file =====
def process(filename, doc_id):
    try:
        with open(filename, 'r', encoding='utf-8', errors='ignore') as
file:
            for line in file:
                parsetoken(line, doc_id)
    except IOError:
        print(f"Error in file {filename}")
        return False
    return True

# ===== MODIFIED: Walk directory =====
def walkdir(cur, dirname):
    global documents
    all_files = []

    for root, dirs, files in os.walk(dirname):
        for file in files:
            all_files.append(os.path.join(root, file))

    # Process each file
    for filepath in all_files:
        documents += 1
        doc_id = documents

        # Store document info
        cur.execute("INSERT INTO DocumentDictionary VALUES (?, ?)",
(filepath, doc_id))
        all_docs.add(doc_id)

        # Process the file
        process(filepath, doc_id)

        # Print progress every 10 files
        if documents % 10 == 0:
            print(f"Processed {documents} documents...")

    return True

# ===== MODIFIED: Calculate TF-IDF =====
def calculate_tf_idf(N):
    """Calculate TF-IDF weights for all term-document pairs"""
    tf_idf_weights = {}

    for term, doc_freq_dict in term_doc_freq.items():
        df = len(term_doc_set[term]) # Document frequency

```

```

        if df == 0:
            continue

        # Calculate IDF
        idf = math.log(N / df) if df > 0 else 0

        for doc_id, tf in doc_freq_dict.items():
            # Calculate TF-IDF
            tf_idf = tf * idf

            if term not in tf_idf_weights:
                tf_idf_weights[term] = {}
            tf_idf_weights[term][doc_id] = {
                'tf': tf,
                'idf': idf,
                'tf_idf': tf_idf
            }

        return tf_idf_weights

# ===== MODIFIED: Store in database =====
def store_in_database(cur, tf_idf_weights):
    """Store the processed data in SQLite database"""

    print("Storing data in database...")

    # 1. Store terms in TermDictionary
    for term, term_id in term_to_id.items():
        df = len(term_doc_set[term])
        cur.execute("INSERT INTO TermDictionary VALUES (?, ?, ?)",
                    (term, term_id, df))

    # 2. Store postings with TF-IDF weights
    for term, doc_weights in tf_idf_weights.items():
        term_id = term_to_id[term]
        for doc_id, weights in doc_weights.items():
            cur.execute("""INSERT INTO Posting
                           (TermId, DocId, tfidf, docfreq, termfreq)
                           VALUES (?, ?, ?, ?, ?)""",
                        (term_id, doc_id, weights['tf_idf'],
                         len(term_doc_set[term]), weights['tf']))

    print(f"Stored {len(term_to_id)} terms and their postings")

# ===== MODIFIED: Main function =====
if __name__ == '__main__':
    t2 = time.localtime()
    print("Start Time: %.2d:%.2d" % (t2.tm_hour, t2.tm_min))

    # Update your folder path
    folder =
r"C:\Users\miss_\PycharmProjects\HelloWorld\pythonProject\venv\cacm\cacm"

    # Connect to database
    con = sqlite3.connect("indexer_part2_tfidf.db")
    con.isolation_level = None
    cur = con.cursor()

    # Drop and create tables with enhanced schema

```

```

cur.execute("DROP TABLE IF EXISTS DocumentDictionary")
cur.execute("DROP TABLE IF EXISTS TermDictionary")
cur.execute("DROP TABLE IF EXISTS Posting")

# Create tables with proper schema for TF-IDF
cur.execute("""
    CREATE TABLE DocumentDictionary (
        DocumentName TEXT,
        DocId INTEGER PRIMARY KEY
    )
""")

cur.execute("""
    CREATE TABLE TermDictionary (
        Term TEXT,
        TermId INTEGER PRIMARY KEY,
        DocumentFrequency INTEGER
    )
""")

cur.execute("""
    CREATE TABLE Posting (
        TermId INTEGER,
        DocId INTEGER,
        tfidf REAL,
        docfreq INTEGER,
        termfreq INTEGER,
        PRIMARY KEY (TermId, DocId),
        FOREIGN KEY (TermId) REFERENCES TermDictionary(TermId),
        FOREIGN KEY (DocId) REFERENCES DocumentDictionary(DocId)
    )
""")

# Create indexes
cur.execute("CREATE INDEX IF NOT EXISTS idxTermDict ON
TermDictionary(Term)")
cur.execute("CREATE INDEX IF NOT EXISTS idxPostingTerm ON
Posting(TermId)")
cur.execute("CREATE INDEX IF NOT EXISTS idxPostingDoc ON
Posting(DocId)")

# Walk directory and process files
walkdir(cur, folder)

# Calculate TF-IDF weights
print("\nCalculating TF-IDF weights...")
N = len(all_docs) # Number of documents
tf_idf_weights = calculate_tf_idf(N)

# Store in database
store_in_database(cur, tf_idf_weights)

# Commit and close
con.commit()
con.close()

# Print statistics
print("\n" + "=" * 50)
print("PROCESSING STATISTICS")
print("=" * 50)
print(f"Number of documents processed: {documents}")

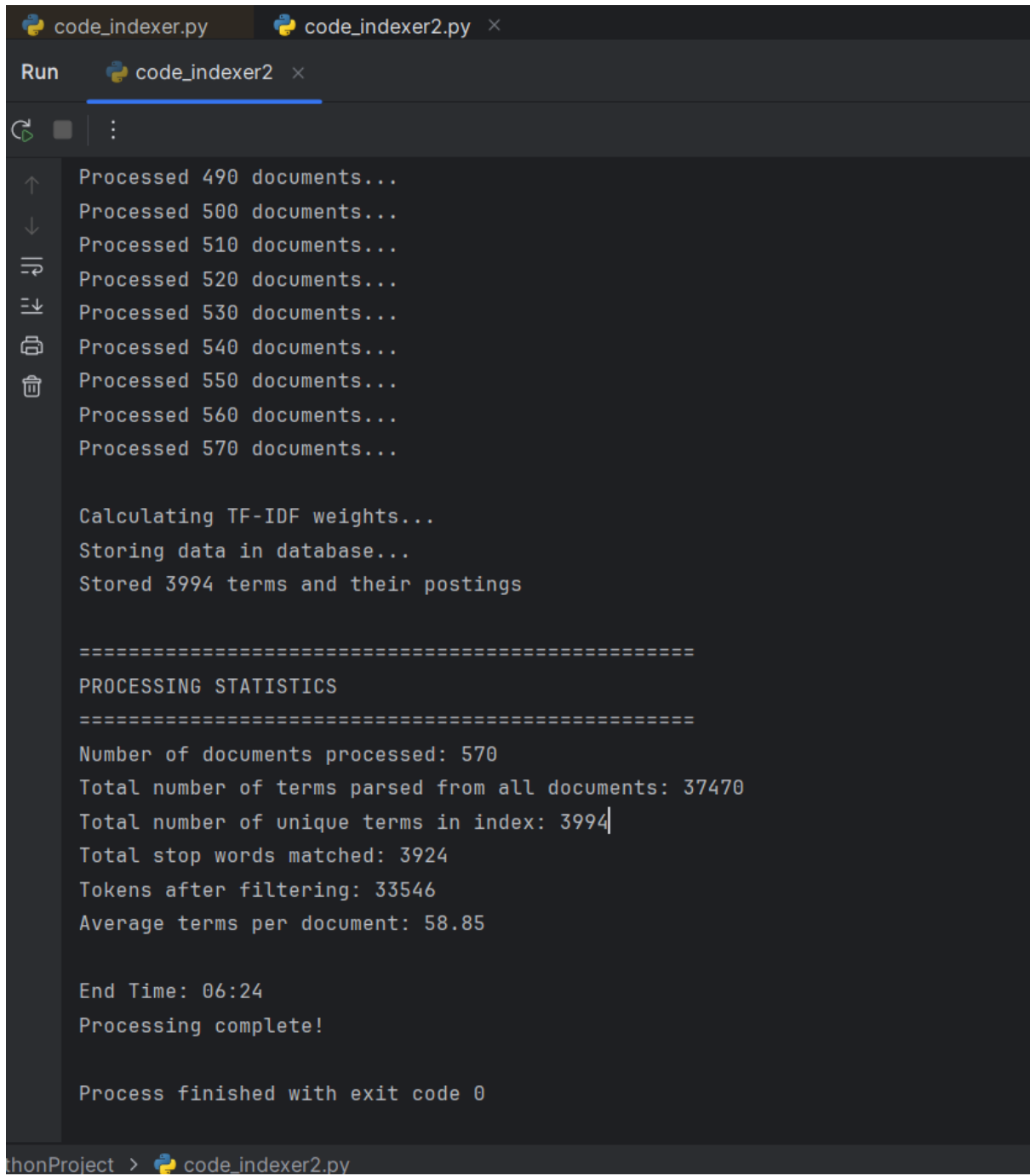
```

```
    print(f"Total number of terms parsed from all documents:
{tokens_before_filtering}")
    print(f"Total number of unique terms in index: {unique_terms}")
    print(f"Total stop words matched: {stop_words_matched}")
    print(f"Tokens after filtering: {tokens_after_filtering}")

# Calculate and print additional statistics
if documents > 0:
    avg_terms_per_doc = tokens_after_filtering / documents
    print(f"Average terms per document: {avg_terms_per_doc:.2f}")

t2 = time.localtime()
print("\nEnd Time: %.2d:%.2d" % (t2.tm_hour, t2.tm_min))
print("Processing complete!")
```

Here is the output of the code:



The screenshot shows a Python IDE with two tabs: `code_indexer.py` and `code_indexer2.py`. The `code_indexer2.py` tab is active, and the output of the script is displayed in the console. The output shows the script processing 570 documents, calculating TF-IDF weights, and storing data in a database. It also displays processing statistics, including the number of documents processed, total terms parsed, unique terms in the index, stop words matched, tokens after filtering, and average terms per document. The script ends with the message "Processing complete!" and "Process finished with exit code 0".

```
code_indexer.py code_indexer2.py x
Run code_indexer2 x
Processed 490 documents...
Processed 500 documents...
Processed 510 documents...
Processed 520 documents...
Processed 530 documents...
Processed 540 documents...
Processed 550 documents...
Processed 560 documents...
Processed 570 documents...

Calculating TF-IDF weights...
Storing data in database...
Stored 3994 terms and their postings

=====
PROCESSING STATISTICS
=====
Number of documents processed: 570
Total number of terms parsed from all documents: 37470
Total number of unique terms in index: 3994
Total stop words matched: 3924
Tokens after filtering: 33546
Average terms per document: 58.85

End Time: 06:24
Processing complete!

Process finished with exit code 0
thonProject > code_indexer2.py
```